Ollama Experiments ✨

Caveats

Ollama

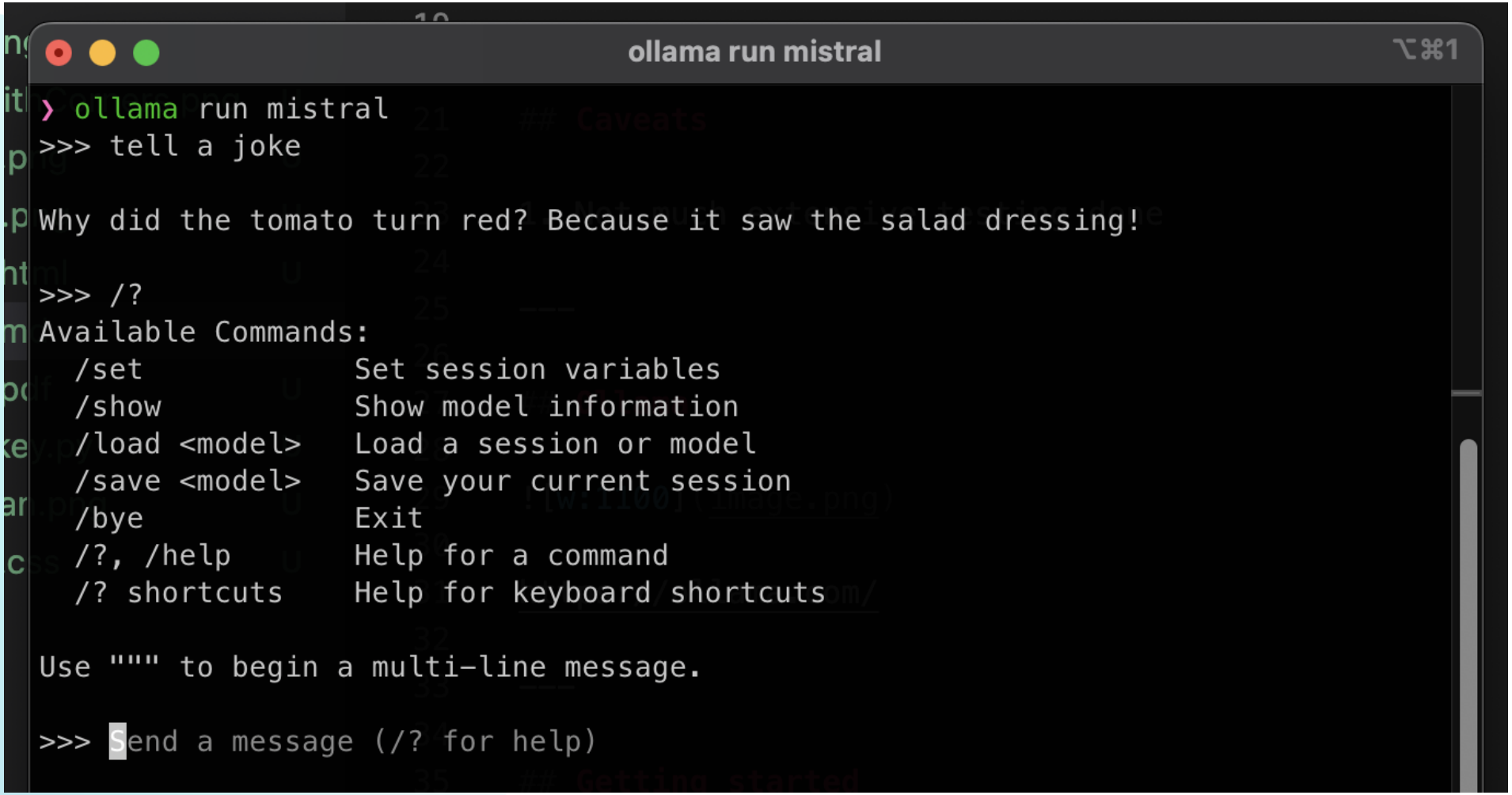


Get up and running with large language models, locally.

Run Llama 2, Code Llama, and other models.
Customize and create your own.

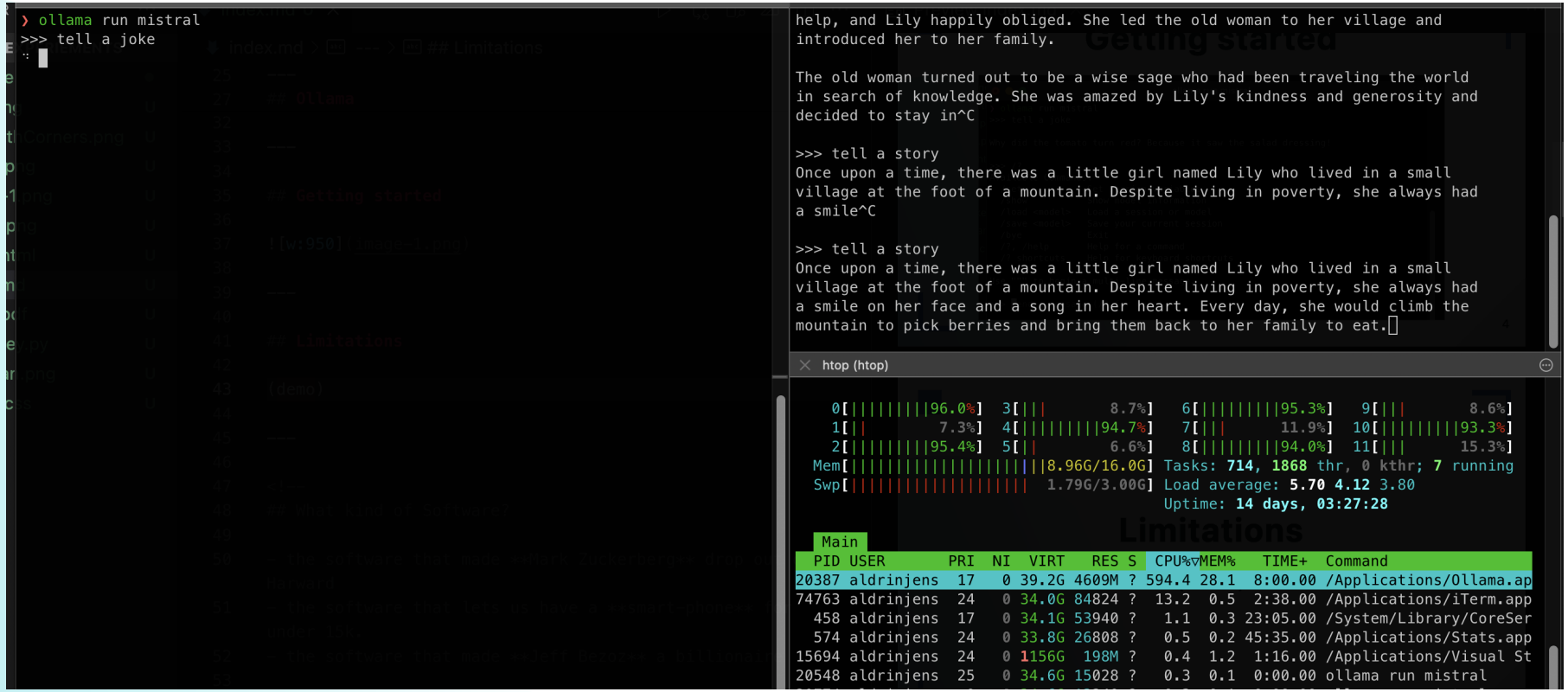
https://ollama.com/

Getting started



```
❯ curl http://localhost:11434/api/generate -d '{
  "model": "mistral",
  "prompt": "Why is the sky blue?"
}'
{"model":"mistral","created_at":"2024-03-07T07:50:03.396958Z","response":"\n","done":false}
{"model":"mistral","created_at":"2024-03-07T07:50:03.564544Z","response":"The","done":false}
{"model":"mistral","created_at":"2024-03-07T07:50:03.717583Z","response":" sky","done":false}
{"model":"mistral","created_at":"2024-03-07T07:50:03.869296Z","response":" appears","done":false}
{"model":"mistral","created_at":"2024-03-07T07:50:04.019765Z","response":" blue","done":false}
{"model":"mistral","created_at":"2024-03-07T07:50:04.180242Z","response":" due","done":false}
{"model":"mistral","created_at":"2024-03-07T07:50:04.347168Z","response":" to","done":false}
{"model":"mistral","created_at":"2024-03-07T07:50:04.508417Z","response":" a","done":false}
{"model":"mistral","created_at":"2024-03-07T07:50:04.680327Z","response":" phenomenon","done":false}
{"model":"mistral","created_at":"2024-03-07T07:50:04.840406Z","response":" called","done":false}
{"model":"mistral","created_at":"2024-03-07T07:50:04.997111Z","response":" Ray","done":false}
{"model":"mistral","created_at":"2024-03-07T07:50:05.163145Z","response":"le","done":false}
```

Limitations



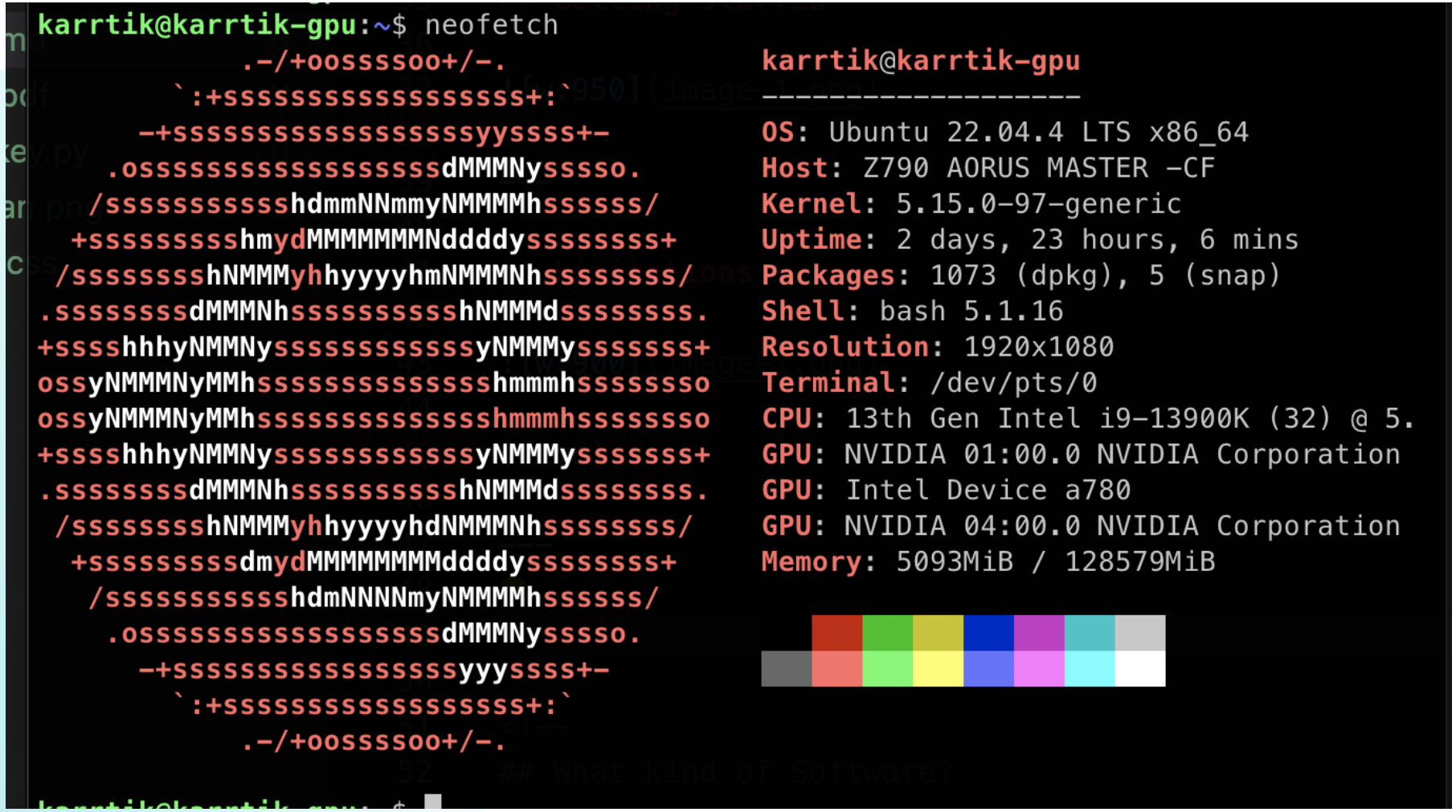
(demo)

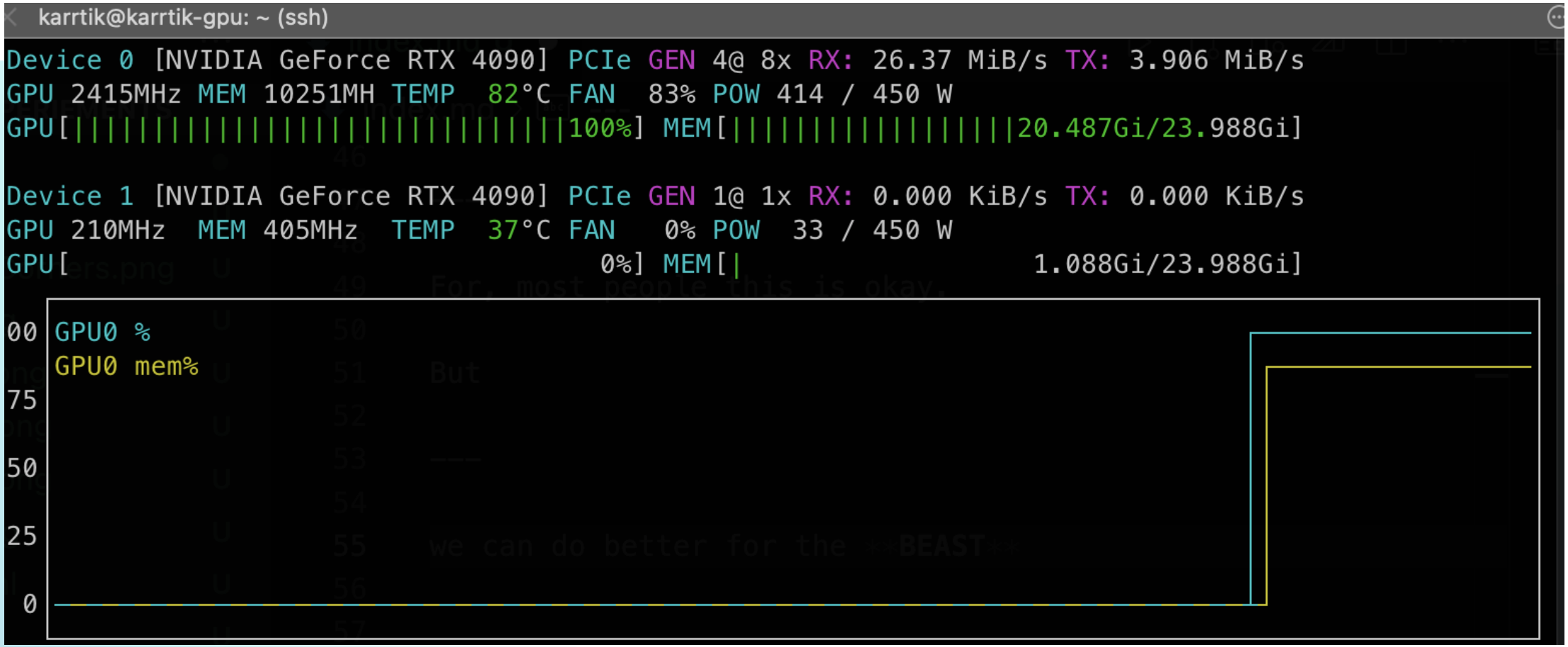


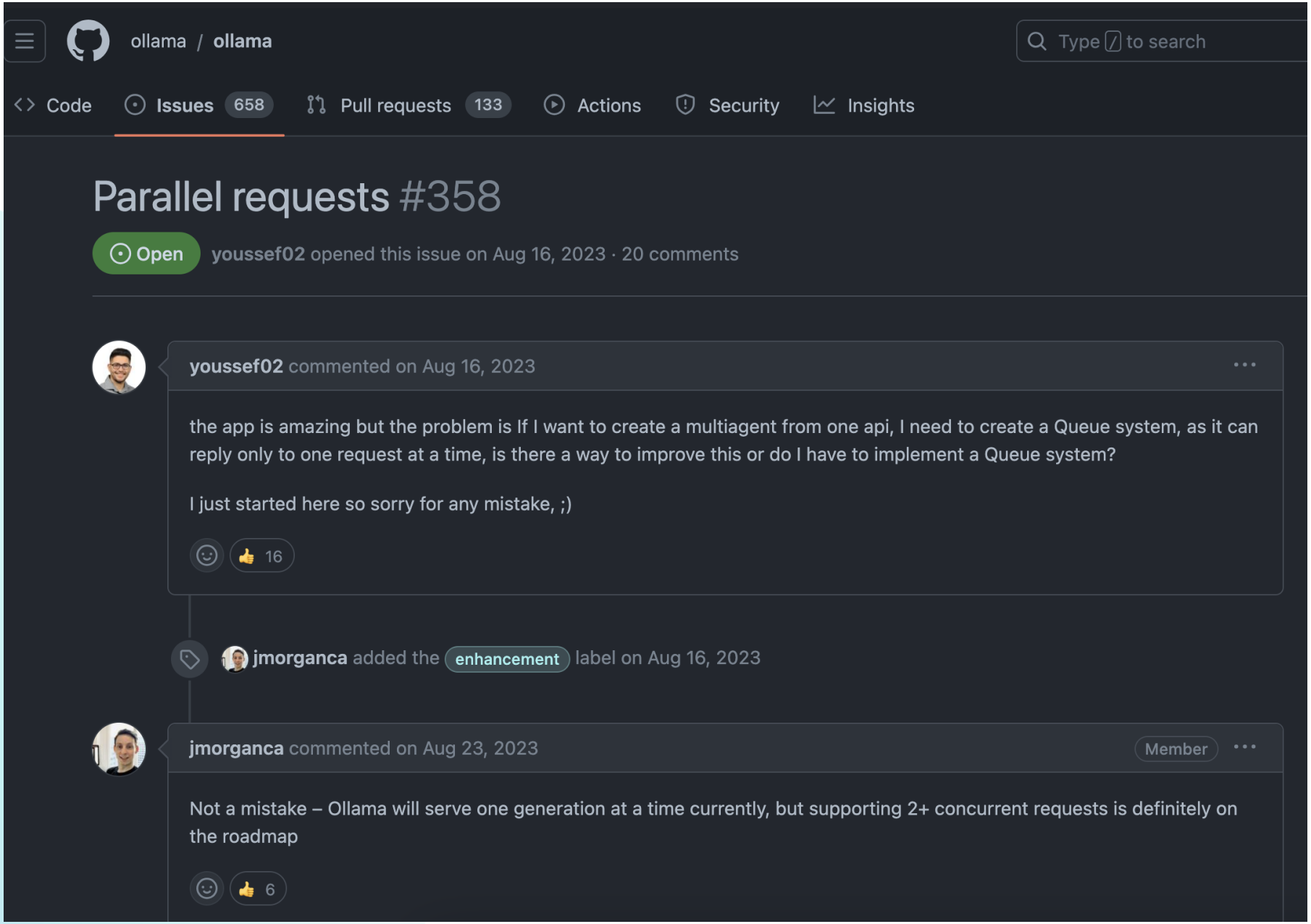
For, most people this is okay.

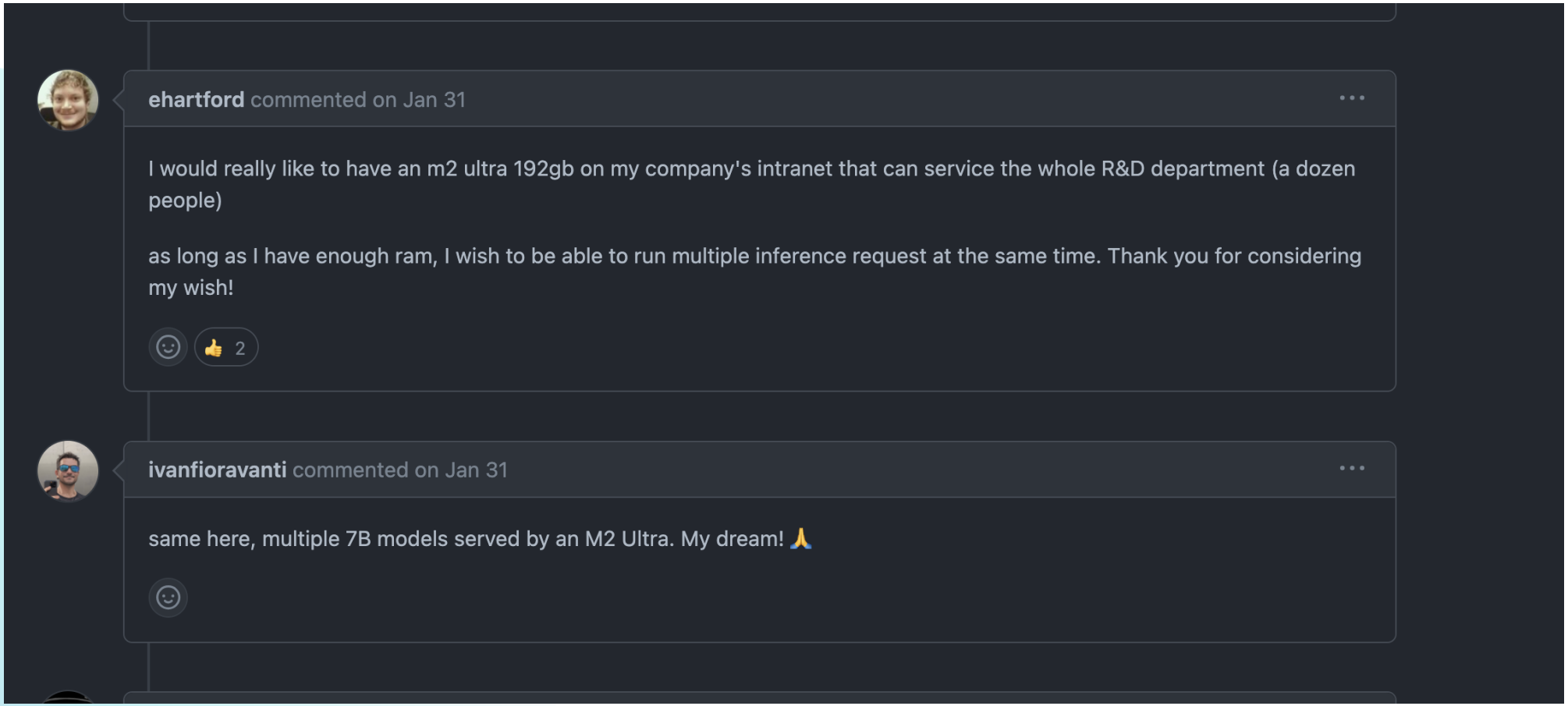
But

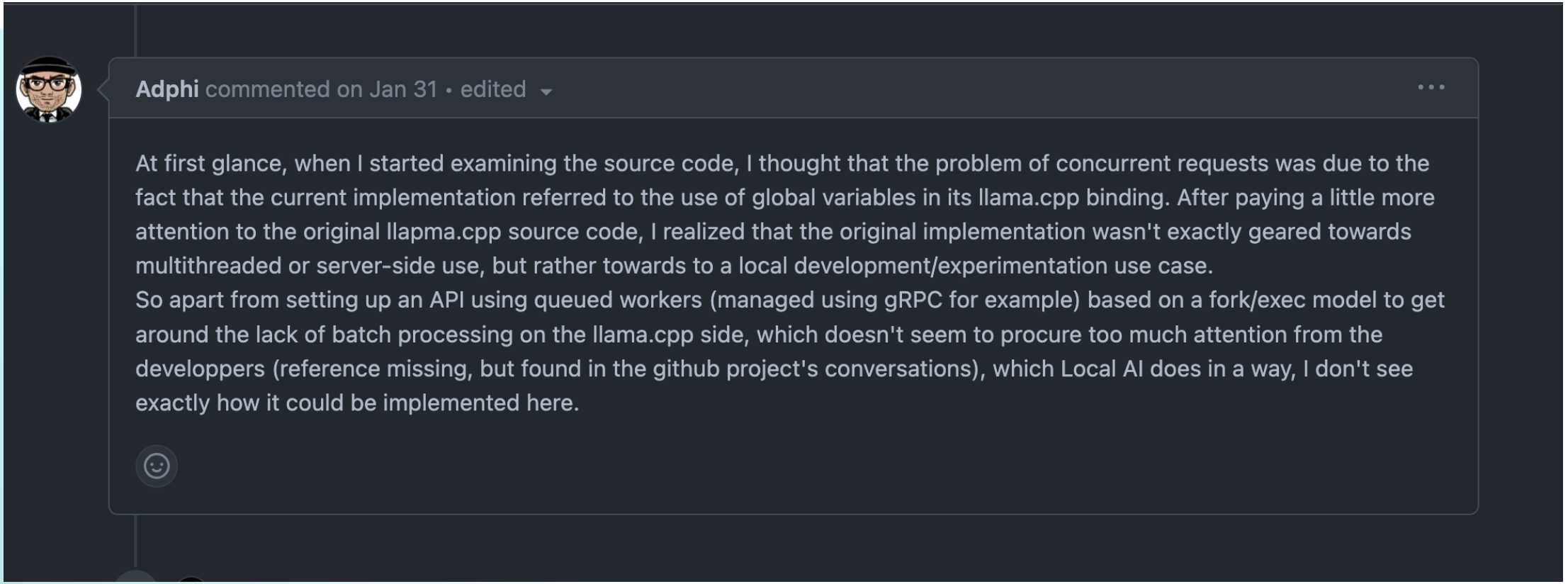
We can do better for the **BEAST**




karrtik@karrtik-gpu: ~ (ssh)
Device 0 [NVIDIA GeForce RTX 4090] PCIe GEN 4@ 8x RX: 26.37 MiB/s TX: 3.906 MiB/s
GPU 2415MHz MEM 10251MH TEMP 82°C FAN 83% POW 414 / 450 W
GPU[|||||||||||||||||||||||||||100%] MEM[||||||||||||||||20.487Gi/23.988Gi]
Device 1 [NVIDIA GeForce RTX 4090] PCIe GEN 1@ 1x RX: 0.000 KiB/s TX: 0.000 KiB/s
GPU 210MHz MEM 405MHz TEMP 37°C FAN 0% POW 33 / 450 W
GPU[0%] MEM[| 1.088Gi/23.988Gi]
00
75
50
25
0
GPU0 %
GPU0 mem%


ollama / ollama
Type / to search
Code
Issues 658
Pull requests 133
Actions
Security
Insights
Parallel requests #358
Open
youssef02 opened this issue on Aug 16, 2023 · 20 comments
youssef02 commented on Aug 16, 2023
the app is amazing but the problem is If I want to create a multiagent from one api, I need to create a Queue system, as it can reply only to one request at a time, is there a way to improve this or do I have to implement a Queue system?
I just started here so sorry for any mistake, ;)
16
jmorganca added the enhancement label on Aug 16, 2023
jmorganca commented on Aug 23, 2023
Member
Not a mistake – Ollama will serve one generation at a time currently, but supporting 2+ concurrent requests is definitely on the roadmap
6


ehartford commented on Jan 31
I would really like to have an m2 ultra 192gb on my company's intranet that can service the whole R&D department (a dozen people)
as long as I have enough ram, I wish to be able to run multiple inference request at the same time. Thank you for considering my wish!
2
ivanfioravanti commented on Jan 31
same here, multiple 7B models served by an M2 Ultra. My dream! 🙏


Adphi commented on Jan 31 • edited
At first glance, when I started examining the source code, I thought that the problem of concurrent requests was due to the fact that the current implementation referred to the use of global variables in its llama.cpp binding. After paying a little more attention to the original llapma.cpp source code, I realized that the original implementation wasn't exactly geared towards multithreaded or server-side use, but rather towards to a local development/experimentation use case.
So apart from setting up an API using queued workers (managed using gRPC for example) based on a fork/exec model to get around the lack of batch processing on the llama.cpp side, which doesn't seem to procure too much attention from the developpers (reference missing, but found in the github project's conversations), which Local AI does in a way, I don't see exactly how it could be implemented here.

Docker to the rescue

```
docker swarm init
```

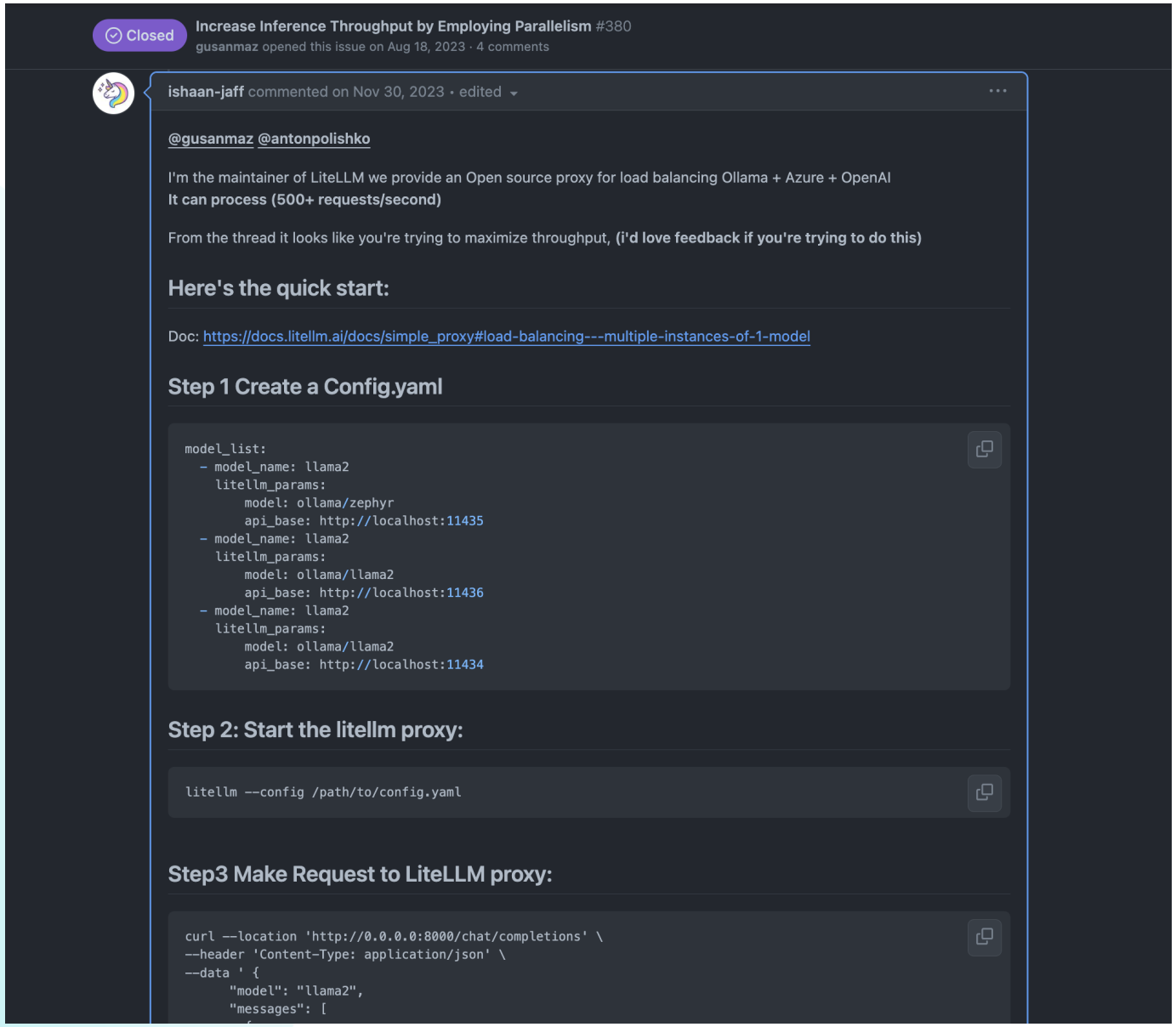
GPU Version

```
docker service create --replicas 10 \
--name ollama \
--constraint 'node.labels.gpu'==true \
--mount type=volume,source=ollama,target=/root/.ollama \
--publish published=11434,target=11434 \
ollama/ollama
```

CPU only version

```
docker service create --replicas 3 \
--name ollama \
--mount type=volume,source=ollama,target=/root/.ollama \
--publish published=11434,target=11434 \
ollama/ollama
```

Workaround



Closed **Increase Inference Throughput by Employing Parallelism** #380

gusanmaz opened this issue on Aug 18, 2023 · 4 comments

ishaan-jaff commented on Nov 30, 2023 • edited

@gusanmaz @antonpolishko

I'm the maintainer of LiteLLM we provide an Open source proxy for load balancing Ollama + Azure + OpenAI
It can process (500+ requests/second)

From the thread it looks like you're trying to maximize throughput, **(i'd love feedback if you're trying to do this)**

Here's the quick start:

Doc: https://docs.litellm.ai/docs/simple_proxy#load-balancing---multiple-instances-of-1-model

Step 1 Create a Config.yaml

```
model_list:
  - model_name: llama2
    litellm_params:
        model: ollama/zephyr
        api_base: http://localhost:11435
  - model_name: llama2
    litellm_params:
        model: ollama/llama2
        api_base: http://localhost:11436
  - model_name: llama2
    litellm_params:
        model: ollama/llama2
        api_base: http://localhost:11434
```

Step 2: Start the litellm proxy:

```
litellm --config /path/to/config.yaml
```

Step3 Make Request to LiteLLM proxy:

```
curl --location 'http://0.0.0.0:8000/chat/completions' \
--header 'Content-Type: application/json' \
--data ' {
      "model": "llama2",
      "messages": [
```

karrtik@karrtik-gpu: ~/dockerswarm (ssh)

```
#!/bin/bash

# create.sh - to create a given set of docker containers running llama2


CONFIG_FILE="config.yml"
VOLUME_NAME="ollama_volume"
MODEL="llama2"

echo "model_list:" > $CONFIG_FILE

# Define function to create and manage ollama containers
create_ollama_container() {
    local container_name="ollama$1"
    local port=$2

    # Check if the container with the same name exists, if yes, remove it
    if docker ps -a --format '{{.Names}}' | grep -q "^$container_name$"; then
        echo "Removing existing container: $container_name"
        docker rm -f $container_name
    fi

    # Check if the container on the specified port exists, if yes, remove it
    if docker ps -a --format '{{.Ports}}' | grep -q "0.0.0.0:$port->"; then
        echo "Removing existing container on port $port"
        docker rm -f $(docker ps -a --format '{{if (index (split .Ports "/") 0) | (index (split . "->") 0) | (eq "'"0.0.0.0:$port"'")}}{{.Names}}{{end}}')
    fi

        # Create and run the new container
    docker run -d --gpus=all \
        -v $VOLUME_NAME:/root/.ollama \
        -p $port:11434 \
        --name $container_name \
        ollama/ollama

    echo "Created container: $container_name on port $port"


    # Add entry to the config file
    echo "  - model_name: $MODEL" >> $CONFIG_FILE
    echo "    litellm_params:" >> $CONFIG_FILE
    echo "        model: ollama/$MODEL" >> $CONFIG_FILE
    echo "        api_base: http://localhost:$port" >> $CONFIG_FILE

}

# Create n different ollama containers with unique names and ports
for i in {1..6}; do
    port=$((11434 + i))
    create_ollama_container $i $port
done

~
~
                                                                    1,1           All
```

karrtik@karrtik-gpu: ~/dockerswarm (ssh)

```yaml
model_list:
  - model_name: llama2
    litellm_params:
        model: ollama/llama2
        api_base: http://localhost:11435
  - model_name: llama2
    litellm_params:
        model: ollama/llama2
        api_base: http://localhost:11436
  - model_name: llama2
    litellm_params:
        model: ollama/llama2
        api_base: http://localhost:11437
  - model_name: llama2
    litellm_params:
        model: ollama/llama2
        api_base: http://localhost:11438
```

config.yml

karrtik@karrtik-gpu: ~/dockerswarm (ssh)

```
#!/bin/bash

# test.sh - to make request to a given  number of llama2 instances

rm file.txt && echo "file.txt cleared"

# Define the command to be executed
curl_command="curl --location 'http://0.0.0.0:8000/chat/completions' --header 'Content-Type: application/json' --data ' {
  \"model\": \"llama2\",
  \"messages\": [
    {
      \"role\": \"user\",
      \"content\": \"tell a long story about unicorns\"
    }
  ]
}' | jq .choices[0].message.content  >> file.txt"

# Run the command n times in parallel
for i in {1..3}; do
    eval "$curl_command" && echo "Command $i succeeded." &  # Run the command in the background
done

# Wait for all background processes to finish
wait

echo "All commands completed."

~
~
~
```

test.sh

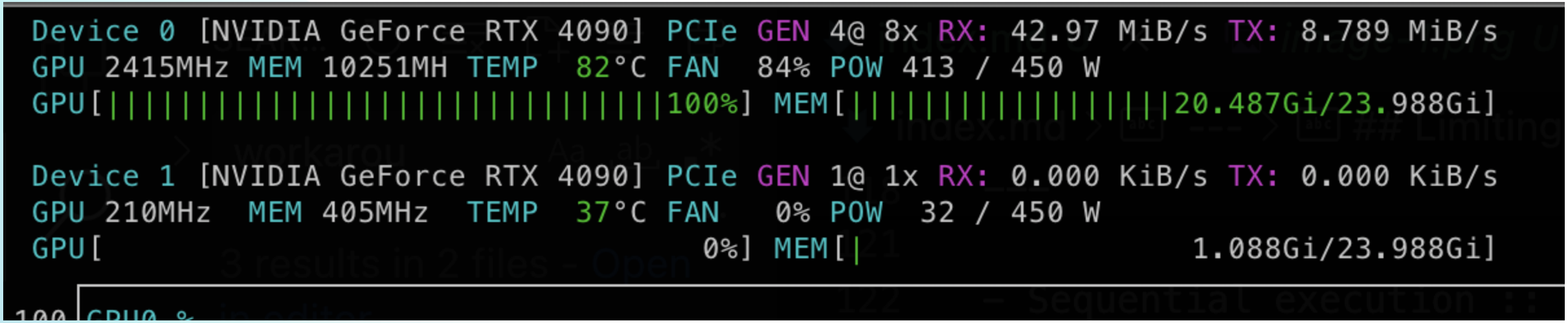
Observations

For 6 llama2 instances

- Sequential execution :: 41+ seconds.
- Parallel execution :: 18+ seconds.

Limiting factor

$$\text{Max Models} = \frac{\text{Max GPU RAM per GPU Stick}}{\text{Max Memory Needed per Model}}$$


Device 0 [NVIDIA GeForce RTX 4090] PCIe GEN 4@ 8x RX: 42.97 MiB/s TX: 8.789 MiB/s
GPU 2415MHz MEM 10251MH TEMP 82°C FAN 84% POW 413 / 450 W
GPU[|||||||||||||||||||||||||||100%] MEM[||||||||||||||||||20.487Gi/23.988Gi]
Device 1 [NVIDIA GeForce RTX 4090] PCIe GEN 1@ 1x RX: 0.000 KiB/s TX: 0.000 KiB/s
GPU 210MHz MEM 405MHz TEMP 37°C FAN 0% POW 32 / 450 W
GPU[0%] MEM[| 1.088Gi/23.988Gi]
100

Still room to improve

Thank You ✨

Resources

- www.gnu.org/gnu/manifesto.html
- www.debian.org/
- www.fsf.org/
- www.youtube.com/watch?v=vwG5N8SCmZ8
- indiafoss.net/2023
- en.wikipedia.org/wiki/Free_software_movement
- slides.aldrinjenson.tech/introduction-to-foss/

May the FOSS be with you ⚔️

This presentation was built using Free Software 🤗